GENESIS 2.5: optimisation and opt-in OpenCL/CUDA acceleration for the GENESIS/PGENESIS compartmental neural simulator

Karol Chlasta[a,*], Grzegorz Marcin Wójcik[b]

[a]*Kozminski University, ul. Jagiellońska 57/59, 03-301 Warsaw, Poland; WarsawIQ, Al. Jana Pawła II 43A/37B, 01-001 Warsaw, Poland*
[b]*Maria Curie-Skłodowska University, ul. Akademicka 9, 20-033 Lublin, Poland*

Abstract

GENESIS is a long-established compartmental neural simulator. We release GENESIS 2.5, adding opt-in OpenCL and CUDA backends for the Hodgkin–Huxley channel-update path and `hines_tree_eliminate`, a GPU Hines tree-elimination kernel for dendritic trees, leaving every existing model, script and MPI workflow unchanged. On cluster hardware the accelerator reaches 57× end-to-end speedup on a 160 000-compartment population, matching the fp64 CPU solver to $\sim 10^{-7}$ V. Measured against other simulators on one node, it runs the Vogels–Abbott COBAHH benchmark 2.3× faster than CoreNEURON on a single CPU core, and on the GPU it overtakes the GPU-native Arbor beyond 18–64 ms of simulated time depending on the card; CoreNEURON's GPU stays 1.23× ahead on that spiking benchmark. Synaptically coupled spiking networks reach the kernels for the first time: GENESIS allowed one spike generator per solver, and lifting that lets such a network be built as one solver per layer, 1.5× faster on the CPU alone. The release also makes $O(n^2)$ model construction linear and repairs seven defects, three of them in the solver GENESIS 2.4 ships.

Keywords: GENESIS, PGENESIS, OpenCL, CUDA, GPU acceleration, compartmental neural simulation, Hines solver

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v2.5 (2026-07-25; OpenCL + CUDA-validated, multi-compartment GPU Hines solve)
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/WarsawIQ/genesis-2.5` (archived release: `https://doi.org/10.5281/zenodo.21658389`)
C3	Legal Code License	GNU General Public License v2 or later (program); GNU Lesser General Public License v2.1 or later (library portions) – see `LICENSE`
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	C, OpenCL C, CUDA C++ (accelerator backends), MPI (PGENESIS), Python (benchmark analysis/plotting)
C6	Compilation requirements, operating environments & dependencies	Linux x86_64, GCC, GNU make, an OpenCL 1.2+ runtime (tested: ROCm 6.3.1 and Mesa rusticl) or a CUDA 12.x toolkit (tested: CUDA 12.8, `sm_80` NVIDIA A100 and `sm_86` NVIDIA A40 on the UMCS cluster) for the GPU backends, an MPI implementation for PGENESIS (MPICH/HYDRA or Open MPI tested on the UMCS cluster)
C7	If available Link to developer documentation/manual	`README.md` and `paper/docs/REPLICATION.md` in the repository
C8	Support email for questions	karol@chlasta.pl

Table 1: Code metadata (mandatory)

Required Metadata

Current code version

Main text

*Corresponding author.
Email addresses: `karol@chlasta.pl` (Karol Chlasta), `gmwojcik@umcs.edu.pl`

1. Motivation and significance

Compartmental simulators remain essential where channel kinetics and dendritic geometry cannot be abstracted away. GENESIS [1] is among the longest-maintained simulators in this class, with PGENESIS extending it to MPI; both are still used in production workflows, scripted in SLI, independent of newer ecosystems.

Modern workstations ship capable integrated GPUs, but GENESIS has no accelerator backend: every channel-kinetics update runs on the CPU whatever hardware is present. A GENESIS or PGENESIS user has no path to accelerator support that does not begin with abandoning their models and scripts.

Existing accelerated simulators, and what they ask of a user. Compartmental simulation on GPUs is not new, and the alternatives are mature. NEURON [11] is the most widely used simulator in this class; CoreNEURON [8] is its optimised compute engine, which strips the interpreter and runs the same models on GPUs through OpenACC, reaching large speedups on multi-rank and multi-GPU systems. Arbor [7] was written from scratch for many-core and GPU hardware, with a performance-portable back end and its own model description. For network models of point neurons rather than detailed morphologies, NEST [12] targets large-scale distributed simulation, and Brian 2 [13] generates code from equations, with GPU execution available through GeNN [14].

Each of these is a strong choice for a new model. None is a path for an existing GENESIS model. CoreNEURON accelerates NEURON's models, not GENESIS's; Arbor and Brian 2 require the model to be re-expressed in their own descriptions; and a GENESIS user's investment is typically in SLI scripts, `.p` cell files and channel prototypes accumulated over years. Porting is not a mechanical translation — as this work found, two implementations of the same published benchmark differ enough in channel kinetics and connectivity construction that establishing their equivalence is itself experimental work (Section 3). The gap GENESIS 2.5 addresses is therefore not "fastest simulator" but "acceleration without migration": the same models and scripts, unchanged, on the hardware the user already has.

Contributions. This work contributes:

1.1 Opt-in OpenCL and CUDA backends for the `hsolve` channel-update path, behind one entry point, leaving every existing CPU,

(Grzegorz Marcin Wójcik)

MPI, model and SLI-script workflow unchanged.

1.2 `hines_tree_eliminate`, a GPU tridiagonal (Hines [10]) elimination kernel extending acceleration from isopotential cells to axially-coupled dendritic trees, running the identical elimination the CPU solver does.

1.3 Seven correctness fixes to released code, none of which requires a user to change a model. Three are in the Hines solver GENESIS 2.4 distributes and reach every multi-neuron `hsolve`, accelerator or not: `inject`-driven voltage transients were silently suppressed for all but the first neuron. Four are in the accelerator as first released, where a model using synaptic channels, a `spike` element, GHK or concentration pools could be dispatched to the GPU and returned wrong voltages with no error raised; such models are now detected at `SETUP` and computed on the CPU. A published network model is what brought these to light, where the synthetic benchmarks had not.

1.4 A latent $O(n^2)$ model-construction bottleneck, unrelated to accelerator support, reduced to linear.

1.5 A head-to-head evaluation against NEURON, CoreNEURON and Arbor, on a published network model and on a multi-compartment population, with the competing simulators' GPU backends included and the equivalence of the independent implementations verified rather than assumed.

Why a GPU helps here is specific to the workload. In a Hodgkin–Huxley model the per-compartment gating integration dominates the cost and is embarrassingly parallel: each compartment advances from its own voltage and its own table lookups, uncoupled from every other within the step. The state is small and the arithmetic intensity low, so the work maps onto thousands of lightweight threads. That makes the channel-update path the natural first target, ahead of the tridiagonal solve.

We provide both backends because they buy different things. OpenCL is vendor-neutral and runs on the integrated GPUs shipped with ordinary workstations, including devices without double precision, which is why the kernel is fp32 throughout. CUDA targets the datacenter cards where most cluster time is available and the profiling tools are standard. Both sit behind one entry point, so the choice is a build flag and neither the model nor its SLI script changes.

We name this release "2.5" rather than "3.0" deliberately: "GENESIS 3" [2] produced independent successor lines (Neurospaces/Heccer [3],

MOOSE) evolving into the CBI federation [4, 5, 6], not a single artifact comparable to GENESIS 2.4. This release is not a re-architecture.

2. Software description

   2.1 Software architecture

   GENESIS 2.5 leaves the existing architecture (SLI interpreter, `hsolve` solver, PGENESIS MPI partitioning) unmodified and adds one component: an OpenCL backend invoked from `hsolve` when an attached element uses `chanmode=4` (or 5) with real ion-channel state. Two dispatch modes are implemented:

   - *Per-step dispatch* (`ocl_chip_update`): one `clEnqueueNDRangeKernel` call per simulation step, with channel state uploaded and downloaded every step.
   - *Multiloop dispatch* (`chip_channel_multiloop`, via `GENESIS_OCL_MULTILOOP=<K>`): a single kernel dispatch runs all K steps in an inner GPU-side loop, amortising the per-step PCIe transfer cost that otherwise dominates wall time (Section 3).

   Every accelerator run emits a non-empty profiling summary, which guards against two silent failures: an `hsolve` with no attached channels never reaching the kernel, and a kernel that fails to build falling back to CPU unnoticed. The kernel is fp32 throughout, so it runs on runtimes without double-precision support; the rest of `hsolve` stays `double`, converting only at the buffer boundary.

   Both dispatch modes update each compartment independently, exact only for single-compartment (isopotential) neurons: a real dendritic tree's compartments are coupled through axial resistance, solved on the CPU by `hsolve` as a compiled bytecode program performing a bottom-up/top-down Gaussian elimination over the tree's tridiagonal system once per step. We add a third kernel, `hines_tree_eliminate` (`ocl_channel.cl`; CUDA port in `cuda/cuda_channel.cuh`), executing this identical bytecode on the GPU, one thread per disconnected tree (neuron), operating on the same flat opcode/coefficient arrays the CPU solver already builds at `SETUP` time. Because independent trees never reference each other's rows (the combined program is block-diagonal), threads need no synchronisation. The channel and elimination kernels dispatch back to back on the same queue/stream, so the K-step batch still uploads and downloads state once. `ocl_hsolve.c`/`cuda_hsolve.c` select this path whenever the `hsolve` contains a multi-compartment tree, leaving the original kernel for single-compartment networks.

Synaptically coupled spiking networks reach the kernels in this release. `hsolve` tracked one spike generator per solver, so such a network had to be built as one solver per cell; it now tracks one per compartment, and a whole layer becomes a single solver whose single-compartment solve the channel kernel completes on the device. One boundary remains, and it is a scheduling one rather than a kernel one: a network with zero axonal delay exchanges spikes every step, so multiloop batching does not apply and each step pays a dispatch. Section 3 measures what that costs.

2.2 Software functionalities

- Unmodified serial (`genesis`), headless (`nxgenesis`), and MPI (PGENESIS) execution paths.
- OpenCL channel-update kernel for Hodgkin–Huxley-style `tabchannel` gating (Na X^3Y^1, K X^4), dispatched per-step or batched via multiloop.
- A CUDA backend at the same call site (`genesis/src/hines/cuda/`): a line-for-line fp32 port behind an identical entry point, so one model and harness drive either backend.
- Device-side dispatch confirmation distinguishing a genuine GPU run from CPU fallback.
- A driven-spiking benchmark (`hh_spiking_benchmark.g`) exercising the repaired `inject` path.
- Three corrected Hines-solver defects (forward-elimination fast-path guard, backward-elimination root handling, `SET_DIAG`/`SKIP_DIAG` row selection), independent of GPU support.
- `hines_tree_eliminate`, a GPU tridiagonal elimination kernel in both backends, extending acceleration to axially-coupled dendritic trees (Section 3).
- Five corrected $O(n^2)$ model-construction defects, restoring linear-time construction independent of GPU support (Section 3).

The tridiagonal kernel decides whether any of this reaches realistic morphologies. Accelerating channel updates alone leaves the axial coupling on the CPU, where the solve becomes the bottleneck as soon as the channel work is taken out of it. Moving it to the device is what lifts acceleration from isopotential point-neuron networks to populations of real dendritic trees, the regime Section 3 measures.

3. Illustrative examples

Reproducing the GPU multiloop result. `genesis/Scripts/benchmark/hh_spiking_benchmark.g` builds N driven Hodgkin–Huxley neurons

under one `hsolve` (`chanmode=4`); `GENESIS_CUDA_MULTILOOP=`K dispatches all K steps in one GPU call. Both arms are wall-clocked identically at $K = 50\,000$ on the A40 and A100.

Table 2 reports both backends after the scheduler and solver fixes. Speedup grows with N throughout, reaching $21.0 \pm 0.3\times$ (CUDA) and $19.2\times$ (OpenCL) on the A40 and $21.9 \pm 0.3\times$ and $20.4\times$ on the A100 at $N = 50\,000$.

Numerical fidelity of the fp32 accelerator. `genesis/Scripts/benchmark/hh1952_ap_verify.g` compares the classic 1952 squid AP across CPU fp64, GPU per-step, and GPU multiloop paths: fp32 agrees with fp64 to $\sim 2 \times 10^{-7}$ V over a single AP, all N neurons identical to $< 10^{-6}$ V (Fig. 1). Over long spiking runs, fp32/fp64 traces drift in spike *phase* but preserve waveform and firing rate, why timing uses driven neurons while correctness uses a single-AP window.

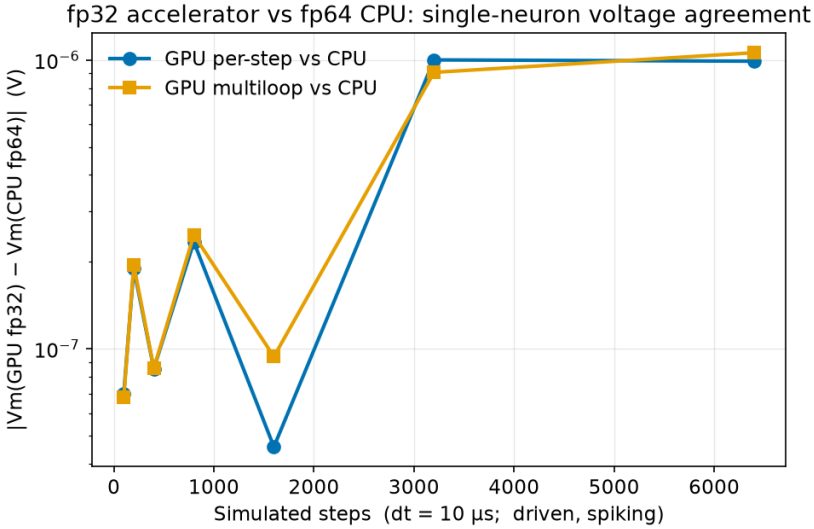

Figure 1: How far the accelerator's single-precision voltages drift from the CPU's double-precision ones, for a driven spiking neuron: under a microvolt after 6400 steps. The two curves are the accelerator's two dispatch modes — one kernel call per step, and one call covering a batch of them — and they lie on each other, so batching costs no accuracy.

CUDA backend on NVIDIA hardware. The CUDA backend builds from the same source (`make USE_CUDA=1`) against the identical harness, and reproduces the fp64 CPU action potential to 7×10^{-8} V, confirming parity with both the CPU and OpenCL paths. Table 2 reports a 10-replicate sweep on the two UMCS cluster GPUs.

These are *end-to-end* speedups, and every figure in this paper uses that one definition. Timing the two arms by different clocks inflates

the ratio by an order of magnitude and can invent differences between cards; measured consistently the A40 and A100 are indistinguishable here ($21.0 \pm 0.3\times$ vs. $21.9 \pm 0.3\times$).
At $N = 500$ the accelerator barely pays for itself: at that size the fixed costs of construction and transfer dominate a run the CPU finishes in under a second.

N	**NVIDIA A40**		**NVIDIA A100**	
	CUDA	**OpenCL**	**CUDA**	**OpenCL**
500	$1.5 \pm 0.1\times$	$1.5\times$	$1.8 \pm 0.3\times$	$1.7\times$
5 000	$8.7 \pm 0.3\times$	$7.5\times$	$9.7 \pm 0.4\times$	$8.7\times$
50 000	$\mathbf{21.0 \pm 0.3}\times$	$19.2\times$	$\mathbf{21.9 \pm 0.3}\times$	$20.4\times$

Table 2: Both accelerator backends (fp32) against the fp64 CPU solver, single-compartment multiloop kernel (`hh_spiking_benchmark.g`, $K = 50\,000$ steps) on the UMCS cluster. End-to-end speedups: both arms of every ratio are wall-clocked around the whole process, so construction, transfers and per-step host work are inside each figure. CUDA is the mean of 10 replicates with propagated uncertainties; OpenCL is the mean of 3. The two cards are close at every size, and OpenCL trails CUDA by roughly 8%, about what a faithful port of the same kernels costs. The OpenCL arm agrees with the fp64 CPU to 3.1×10^{-8} V. Raw data: `cluster_bringup/logs/`.

Multi-compartment GPU acceleration. The multiloop kernel above is exact only for single-compartment (isopotential) neurons: for a real dendritic tree it silently drops all axial coupling. `hines_tree_eliminate` (Section 2.1) performs this elimination on-device, dispatched whenever the `hsolve` contains a multi-compartment tree.
Correctness. Linear-chain and branching benchmarks compare GPU against fp64 CPU across `NCOMP` = 1–32 and up to $N = 10\,000$ neurons: agreement is $\sim 10^{-7}$ V, matching single-compartment fidelity. One topology is not handled: a single-compartment branch off a branch point, which violates the kernel bootstrap's seed-row assumption. Both backends detect it at initialisation and compute that `hsolve` on the CPU, so it costs speed rather than accuracy.
Speed. Table 3 and Fig. 2 report speedup (mean $\pm$ std, 10 replicates) on the two UMCS cards, sweeping $N = 100$–$50\,000$ at `NCOMP` = 16, a range unreachable before the model-construction fix (Impact): $N = 50\,000$ took 23.8 ± 0.3 s to build there, versus not finishing before. Branching topology gives statistically indistinguishable speedups and is omitted from the figure. Dispatch uses $K = 200$ steps ($K = 100$ for $N \geq 5000$), smaller than the single-compartment sweeps' $K = 50\,000$.
Two figures per card answer different questions. The *step-phase* speedup times the simulation loop alone and measures what the kernel can

do: both cards are slower than CPU at $N \leq 500$, pull ahead past $N \approx 500$–1000, and reach $37.3 \pm 0.1\times$ (A40) and $80.8 \pm 1.0\times$ (A100) at $N = 50\,000$, neither plateaued. The *end-to-end* speedup wall-clocks the whole process, which is what a user waits for: it is far lower, $3.62 \pm 0.05\times$ (A40) and $3.90 \pm 0.07\times$ (A100) at $N = 50\,000$. The gap is not a kernel deficiency but Amdahl's law applied to a short run — at $K = 200$ steps the unaccelerated construction phase dominates, and the end-to-end figure climbs toward the step-phase ceiling as K grows. Quoting only the step-phase number would overstate what the accelerator delivers in practice.

The cards nearly coincide end-to-end despite the A100 leading by more than $2\times$ on the step phase — consistent with an fp32 kernel using no tensor cores, the GPU phases taking 7.83 s (A40) against 8.06 s (A100) at $N = 50\,000$. Raw data: `cluster_bringup/logs/weekend_campaign_*_20260816_clean.csv` and `..._multicomp_walltime_createmap_*_clean.csv`.

Parallelisation granularity. `hines_tree_eliminate` assigns one GPU thread per tree, so speedup comes from spreading many neurons across threads, not from parallelising elimination *within* a tree. For a single detailed morphology alone ($N = 1$), only one thread is active: measured directly, the kernel is $\sim$20–21$\times$ *slower* than the CPU at `NCOMP` = 2000 and 8000, stable with tree size as expected for a single-thread bottleneck. The kernel therefore accelerates population-scale simulation (Table 3), not the single-detailed-cell regime.

Population-scale model construction was $O(n^2)$; now $O(n)$. Pushing toward the $\sim$31 000-neuron Blue Brain Project scale [9] exposed a defect independent of GPU support: 527 000 compartments did not finish building within 900 s. Profiling gave a fitted exponent of 2.31, traced to five pre-existing linear-scan-per-element patterns in element creation, path resolution and `hsolve` SETUP, none GPU-specific — each rescanning siblings, paths or compartments where hashing or amortised growth would do. Fixing all five (Fig. 3) gives an exponent of 0.99: that population now builds in 14.6 ± 0.2 s, and 1.7 million compartments in 51.5 ± 0.6 s. Output is byte-identical on every validation script, confirming a pure performance fix; full diagnosis in `genesis/src/hines/GPU_HINES_SOLVE_DESIGN.md`.

The multi-compartment figures are a floor set by the benchmark, not a ceiling set by the kernel. The sweeps above use $K = 200$ ($K = 100$ for $N \geq 5000$), which at $dt = 50\,\mu$s is 5–10 ms of simulated activity, short enough that fixed costs dominate both arms. Repeating the measurement at $N = 10\,000$ over a range of K (Table 4)

		step-phase		end-to-end	
N	**total comps**	**A40**	**A100**	**A40**	**A100**
100	1 600	0.2 ± 0.02	0.3 ± 0.02	0.43 ± 0.05	0.61 ± 0.02
500	8 000	1.1 ± 0.03	1.7 ± 0.11	0.77 ± 0.02	1.04 ± 0.03
1 000	16 000	2.0 ± 0.08	3.1 ± 0.07	1.10 ± 0.08	1.46 ± 0.04
2 000	32 000	4.5 ± 0.39	6.4 ± 0.18	1.37 ± 0.06	1.57 ± 0.05
3 000	48 000	6.7 ± 0.03	12.0 ± 0.37	1.87 ± 0.07	1.89 ± 0.04
5 000	80 000	6.9 ± 0.04	14.5 ± 0.47	2.60 ± 0.06	2.66 ± 0.03
7 000	112 000	9.8 ± 0.05	22.0 ± 0.43	2.92 ± 0.05	3.08 ± 0.03
10 000	160 000	13.7 ± 0.06	30.6 ± 0.68	3.01 ± 0.04	3.23 ± 0.04
15 000	240 000	20.3 ± 0.12	42.3 ± 0.77	3.27 ± 0.05	3.52 ± 0.03
20 000	320 000	25.3 ± 0.12	51.6 ± 1.16	3.39 ± 0.04	3.71 ± 0.04
30 000	480 000	31.9 ± 0.13	66.1 ± 1.12	3.53 ± 0.05	3.87 ± 0.05
40 000	640 000	35.2 ± 0.13	75.5 ± 0.96	3.56 ± 0.04	3.90 ± 0.04
50 000	800 000	37.3 ± 0.07	**80.8 ± 1.01**	3.62 ± 0.05	**3.90 ± 0.07**

Table 3: Multi-compartment GPU tree-elimination (fp32) vs. CPU (fp64), UMCS A40/A100, `NCOMP` = 16, linear-chain, batched multiloop ($K = 200$, $K = 100$ for $N \geq 5000$); mean ± std over 10 replicates. *Step-phase* times the simulation loop alone and measures kernel capability; *end-to-end* wall-clocks the whole process, including model construction. The two arms of the end-to-end measurement are timed identically – see `cluster_bringup/54_multicomp_walltime.sh`. The end-to-end runs build the population with a single `createmap` from a prototype, as network models do, rather than the explicit SLI loop of the original benchmark; the step-phase figures are unaffected by that choice, since they exclude construction. "–" = not measured. Raw data: `cluster_bringup/logs/`.

separates them: end-to-end speedup rises from 4.3× at $K = 200$ to $57.0 \pm 0.9\times$ at $K = 5000$, and step-phase from 13.7× to 116.2×. Both grow because the one-time costs they contain are amortised over more steps: model construction for the end-to-end figure, buffer setup and the single batched dispatch for the step-phase figure.

Fitting fixed and marginal costs separates them: construction takes ≈1.3 s on both arms, while one step costs 2.31×10^{-2} s on the CPU against 1.42×10^{-4} s on the GPU, a ratio of 163×. At $K = 5000$ the GPU still reproduces the CPU voltages to $\sim 10^{-7}$ V, so this is amortisation, not skipped work. Table 3 reports the short-K figures because they are what the standing benchmark measures, but a 250 ms simulation of this population already reaches 57× end-to-end.

Reproducing the PGENESIS scaling result. The existing MPI path is unaffected: a strong-scaling sweep ($N = 2400$ HH neurons, $P = 1$–24 ranks) gives $E_P = 0.95$ at $P = 2$, an efficiency knee near $P \approx 5$, and 3.1–4.0× at $P = 4$–8.

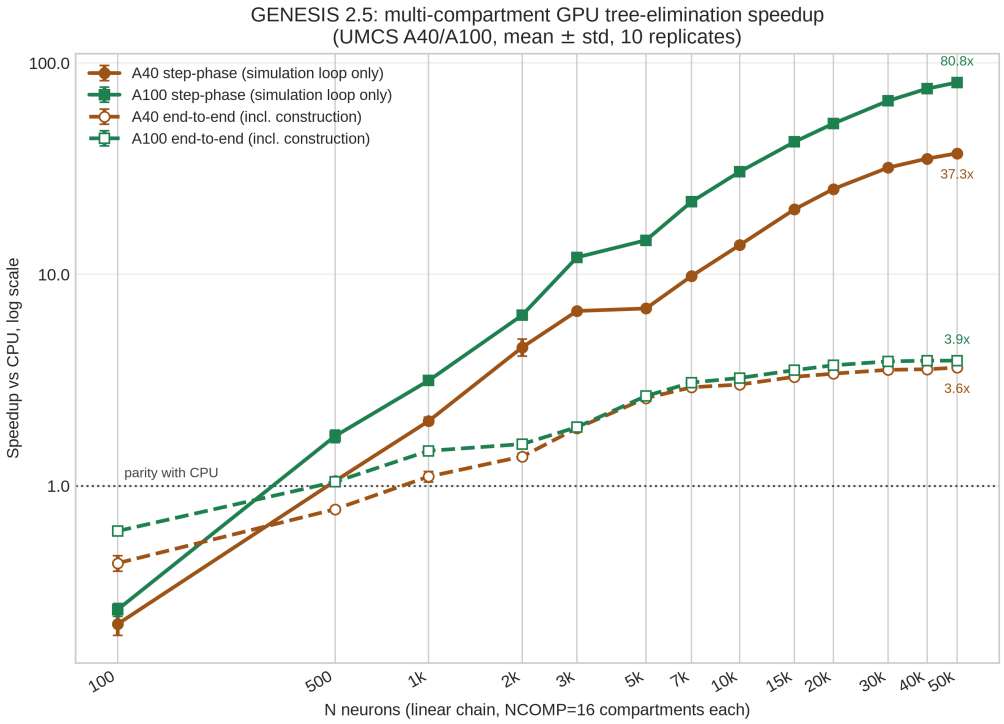

Figure 2: How much the tree-elimination kernel gains over the CPU as the population grows, at 16 compartments per neuron (log-log, mean ± std over 10 replicates). Colour is the card; solid lines time the simulation loop alone, dashed ones the whole run including model construction. The GPU loses below a few hundred neurons and wins past $N \approx 500$–1000, the loop climbing to 80.8× on the A100 and 37.3× on the A40 without levelling off. End-to-end it stops near 3.9×, because these runs are only $K = 200$ steps long and model construction stays on the CPU.

Head-to-head against NEURON and CoreNEURON. The speedups above are internal (GPU vs. CPU within GENESIS) and say nothing about how GENESIS compares with other simulators. We therefore ran the Vogels–Abbott COBAHH network [15, 16] in GENESIS (`Scripts/VAnet2`) and in the reference NEURON implementation (ModelDB 83319, its `NEURON/cobahh` directory) on the same cluster node (Table 5, Fig. 4), with every CPU arm single-threaded. GENESIS 2.5 completes the 5.0 s simulation in 46.9 ± 2.1 s as published, and in 33.2 ± 0.7 s built as one solver per layer, which this release allows for the first time: 2.30× faster than CoreNEURON on the same core, 2.88× faster than NEURON 9.0.2.

CoreNEURON exists to make NEURON models faster, discarding the interpreter and rebuilding the data layout for vectorisation. It is in

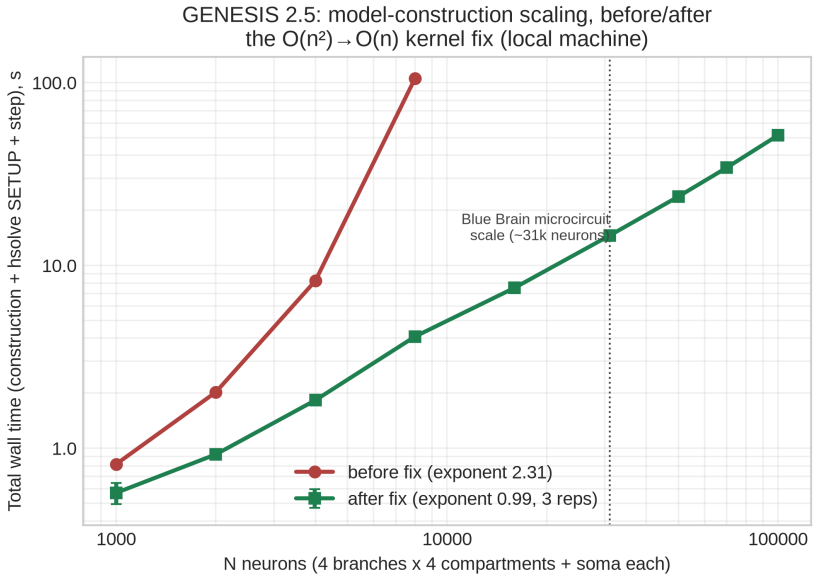

Figure 3: Model-construction wall time vs. N, log-log, before (single replicate) and after (mean $\pm$ std, 3 replicates) the five $O(n^2)$ fixes, extended to $N = 100\,000$. The "before" series stops at $N = 8000$: larger N did not finish within the tested budget.

K steps	CPU (s)	GPU (s)	End-to-end
200	5.86	1.37	4.28 $\pm$ 0.12
500	13.00	1.42	9.18 $\pm$ 0.25
1 000	24.87	1.49	16.73 $\pm$ 0.36
2 000	48.03	1.63	29.48 $\pm$ 0.91
5 000	116.75	2.05	**57.01 $\pm$ 0.88**

Table 4: End-to-end wall clock against simulation length, $N = 10\,000$ neurons $\times$ 16 compartments (160 000 compartments), UMCS A40, mean $\pm$ std over 5 replicates, both arms timed identically. The fixed cost each arm carries, $\approx$1.3 s of model construction, is unchanged across the sweep, so the speedup rises as K grows: at $K = 200$ construction is most of the GPU run, at $K = 5000$ it is under a tenth of it. Re-measured on the same node after the model corrections of Fig. 6 (3 replicates): GPU 2.06 s against 2.05 s and CPU 113.9 s against 116.8 s, giving 55.4×. The difference sits in the CPU arm, which those corrections cannot change the operation count of, and the figure reported here is the fuller 5-replicate campaign. Raw data: `cluster_bringup/logs/multicomp_ksweep_inf02_A40_20260816.csv`, produced by `cluster_bringup/55_multicomp_ksweep.sh`.

turn 1.25× faster than NEURON 9.0.2, the expected ordering, which suggests that arm is configured correctly.

Because these are independent implementations, we verified their equivalence rather than assuming it: every arm has 4000 single-compartment cells with 64 excitatory and 16 inhibitory inputs, is driven externally

for the first 50 ms only, uses $dt = 0.05$ ms for 5.0 s, and settles into the same regime, 26.8 to 30.1 Hz across all four.

The stock benchmark cannot run under CoreNEURON at all, since its channels are ChannelBuilder objects rebuilt inside the interpreter. We transcribed them into NMODL, reproducing the published COBAHH rates, after which CoreNEURON yields a spike train byte-identical to NEURON's. The same mechanisms, compiled into an Arbor catalogue, give the fourth arm: 149.4 ± 1.1 s on the A100 that runs CoreNEURON in 27.0. Arbor cannot express the benchmark's zero axonal delay — it advances in minimum-delay epochs — so that arm uses one timestep and exchanges spikes every step, which is also why our own batched dispatch is unavailable here. Every cross-simulator arm runs on one node, inf03 with its A100: the two cluster nodes carry different CPUs, so single-threaded times taken on different nodes would not be comparable. The GPU arms are measured on both cards (Fig. 5). Harness: `cluster_bringup/`.

CoreNEURON on the GPU. Leaving CoreNEURON on the CPU, where it was never meant to stay, would flatter us. Building its OpenACC backend with NVHPC left NEURON itself unable to start, so we ran CoreNEURON on its own: NEURON writes the model out with `nrncore_write`, and `special-core` reads those files and simulates without it, which keeps NEURON on its working GCC build. On one A100 CoreNEURON completes the same network in 27.0 ± 0.1 s — 2.83× faster than its own CPU arm, and 1.23× faster than our best CPU time. Its spike count differs from the CPU arms by 6% (592 865 against 558 824) and from GENESIS by 10%: CoreNEURON's GPU path requires cell permutation, which reorders synaptic delivery, and the network is chaotic, so counts diverge while the regime does not. This is a throughput comparison in a common regime, not a claim of identical trajectories.

This is the boundary of what the release offers. Built as one solver per layer the model does run on the accelerator, but only at parity with its own CPU arm (Section 5), so CoreNEURON's GPU stays ahead by 1.23×. Where the accelerator pays — multi-compartment populations — the comparison below is GPU on both sides.

Do the three integrate the same model? Measured rather than assumed (Fig. 6): NEURON and Arbor agree to 0.02 mV at the action-potential peak and to 0.01% on firing rate, while GENESIS peaks 4.6 mV lower and fires faster, its Hodgkin–Huxley rates being written about −70 mV where NEURON's use −65. Setting the comparison up corrected three errors in our own harness — Arbor injected into

Simulator	Channels	Device	Wall (s)	Spikes
CoreNEURON 9.0.2	compiled NMODL	A100	27.0 ± 0.1	592 865
GENESIS 2.5, one solver per layer	tabulated	CPU	**33.2 ± 0.7**	545 371
GENESIS 2.5, VAnet2 as published	tabulated	CPU	46.9 ± 2.1	536 600
CoreNEURON 9.0.2	compiled NMODL	CPU	76.5 ± 0.3	558 824
NEURON 9.0.2	compiled NMODL	CPU	95.8 ± 0.2	558 824
NEURON 9.0.2	ChannelBuilder	CPU	123.3	574 138
Arbor 0.10.0	compiled NMODL	A100	149.4 ± 1.1	602 720

Table 5: Vogels–Abbott COBAHH network, 4000 cells, 80 synapses each, $dt = 0.05$ ms, 5.0 s simulated, one UMCS node; CPU arms are single-threaded. Built as one solver per layer, which this release makes possible, GENESIS 2.5 (bold) runs the network 2.30× faster than CoreNEURON on the same core, and only CoreNEURON on a GPU is faster still, by 1.23×. Every arm settles into the same activity regime, 26.8–30.1 Hz, so the wall times cover comparable work. Means ± std over 3 replicates, except the ChannelBuilder row, a single run of the benchmark in its original form. Harness: `cluster_bringup/coreneuron/`.

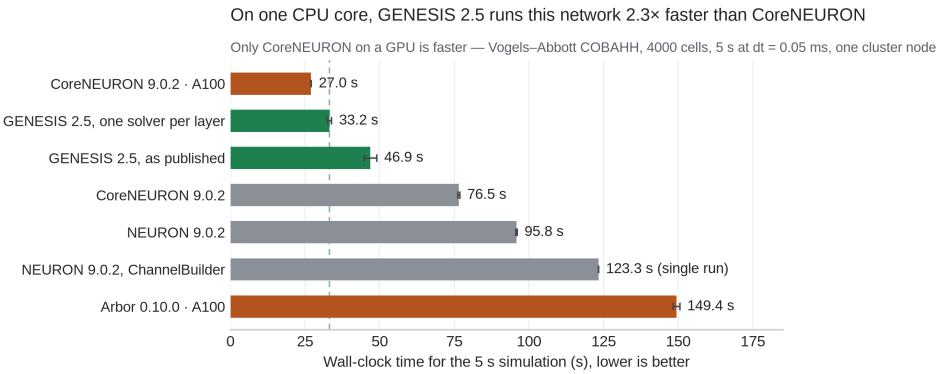

Figure 4: The measurements of Table 5, drawn. Green is GENESIS, orange the arms running on an A100, grey the CPU arms it is compared against; the dashed guide marks the GENESIS time, so every bar reaching past it is slower. Means of 3 replicates with standard deviations, except the ChannelBuilder row. Arbor comes last despite its GPU because zero axonal delay forces it to exchange spikes every step.

the middle of the cable rather than the soma, Arbor took NEURON's default reversal potentials instead of the model's, and GENESIS scaled every compartment's channel conductance by the soma's area — none of which moved a wall time by more than 1.3%. Timing is unaffected otherwise: with no synapses and no events, switching the injection off moves no arm by 2%.

GPU against a GPU-native simulator. In the comparison above the accelerator reaches only parity with its own CPU arm, because a zero-delay spiking network dispatches every step (Section 2.1). To test

it where batching does apply we used a model — $N = 10\,000$ neurons, each a chain of 16 compartments, HH channels, current injection, no synapses — reimplemented in NEURON and in Arbor [7], which was written from scratch for GPU hardware.

Which simulator is faster depends on run length, and the crossover is measurable. Over $K = 5000$ steps on one node (Table 6) Arbor finishes in 1.56 ± 0.03 s against our 1.59 ± 0.01 s; over $K = 50\,000$ the order reverses, 4.56 ± 0.01 s against Arbor's 5.95 ± 0.02 s. Both are linear in K. Fitted over six run lengths from $K = 1000$ to 50 000, GENESIS costs 1.28 s plus 65.6 μs per step and Arbor 1.08 s plus 97.3 μs: our marginal cost is 33% lower while theirs starts 0.20 s ahead, so the lines cross at $K \approx 6400$, or 64 ms of simulated time at $dt = 0.01$ ms (Fig. 5). Below it Arbor wins on setup; above it, where modelling studies sit, GENESIS 2.5 is faster. The A40 gives the same ordering with a wider margin, crossing at $K \approx 1800$. The device profile explains the split: our step phase exceeds kernel time by 0.21 s at both run lengths, so the excess is one-time dispatch setup, not per-step work. This is the cost of the design that keeps existing models working: Arbor generates specialised code per mechanism, while our kernels interpret the `hsolve` opcode program at run time. On single-threaded CPU the GENESIS solver is 2.02× slower than CoreNEURON here, the opposite of Table 5, because the two benchmarks stress different things: the spiking network is dominated by synaptic event handling over single compartments, this one by the tridiagonal solve, where CoreNEURON's vectorised layout is stronger. Either way it is the accelerator, not the solver, that this release contributes. Arbor is also faster on 128 CPU threads than on the GPU at this size, its construction cost dominating. We report no PGENESIS arm: what is compared here is the accelerator.

A reproduction pack is included: `shreproduce/run_all.sh` re-measures every claim here, regenerates the figures from those measurements, and prints each published value beside the reader's own. Raw data and full steps: `paper/docs/REPLICATION.md`.

4. Impact

GENESIS/PGENESIS users with channel-kinetics-heavy models now have a path to GPU acceleration (Section 3) without migrating models, SLI scripts, or MPI workflows — the barrier an architecturally different successor such as MOOSE would impose. The accelerator's scope is deliberate: both kernels interpret the `hsolve` opcode program directly and implement the five opcodes a current-driven `tabchannel` model produces, so compartments needing anything else (synaptic channels, a `spike` element, GHK, concentration pools) are detected at `SETUP`

Simulator	Backend	Wall (s)
$K = 5000$ steps (50 ms simulated)		
Arbor 0.10.0	CPU, 128 threads	1.09 ± 0.04
Arbor 0.10.0	GPU (A100)	**1.56 ± 0.03**
GENESIS 2.5	GPU (A100)	1.59 ± 0.01
CoreNEURON 9.0.2	CPU, 1 thread	60.94 ± 0.15
NEURON 9.0.2	CPU, 1 thread	82.82 ± 2.00
GENESIS 2.5	CPU, 1 thread	123.41 ± 1.75
$K = 50\,000$ steps (500 ms simulated)		
GENESIS 2.5	**GPU (A100)**	**4.56 ± 0.01**
Arbor 0.10.0	GPU (A100)	5.95 ± 0.02

Table 6: Multi-compartment Hodgkin–Huxley benchmark across simulators: $N = 10\,000$ neurons × 16 compartments (160 000 compartments), $dt = 0.01$ ms, all on UMCS node inf03 (Xeon Platinum 8358, NVIDIA A100), mean ± std over 3 replicates, both arms of every pair timed identically. Thread counts differ by simulator and are stated: Arbor uses all cores by default, the others are single-threaded. The CPU arms are reported only at $K = 5000$, where they were measured; extrapolating them to the longer run would be an estimate, not a measurement. The GPU rows come from one sweep of six run lengths, which is also what Fig. 5 fits, so the table and the figure cannot disagree. The models match in geometry, passive properties, conductances, injection and timestep, verified against recorded voltages (Fig. 6); this is a throughput comparison and not a claim that the three produce identical trajectories. Harness and raw data: `cluster_bringup/coreneuron/`.

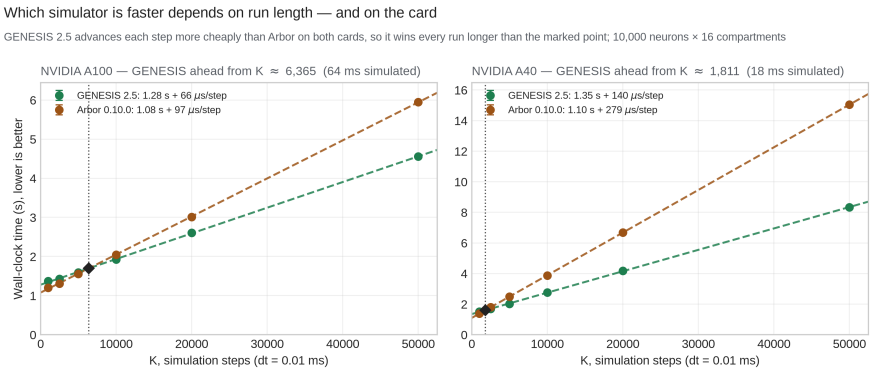

Figure 5: Neither simulator is faster in general: which one wins depends on how long the run is, and where the changeover falls depends on the card. Both are linear in K — Arbor starts about 0.2 s ahead on setup, GENESIS 2.5 advances each step more cheaply — so a comparison at a single run length reports whichever side of the crossing the author happened to choose. Our per-step margin is the wider one on the A40, 140 against Arbor's 279 μs, where on the A100 it is 66 against 97; the crossing moves accordingly, from $K \approx 6400$ to $K \approx 1800$. Our kernel is fp32 where Arbor computes in double, and the A40's double-precision throughput is a fraction of the A100's. $N = 10\,000$ neurons × 16 compartments, mean ± std over 3 replicates.

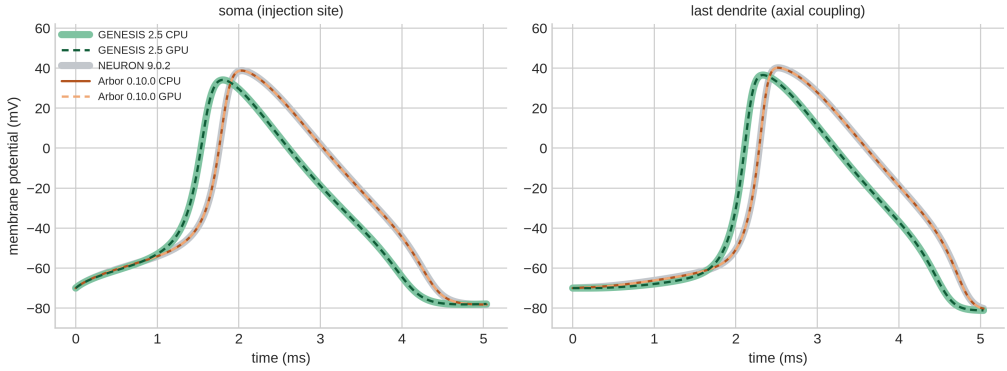

Figure 6: Do the three simulators integrate the same model? One cell's first action potential, at the soma where the current is injected and at the far end of the dendrite, which moves only if axial coupling is integrated. Each simulator is drawn twice — one backend wide and pale, the other dashed on top — so a coinciding pair reads as agreement rather than as a missing line. NEURON and Arbor lie on each other to 0.02 mV at the peak and agree to 0.01% on firing rate. GENESIS peaks 4.6 mV lower because its Hodgkin–Huxley rates are written about a −70 mV rest where NEURON's are written about −65. Our own fp32 and fp64 backends agree to 2.6×10^{-5} V. Data: `cluster_bringup/logs/vmcross/`.

and computed on the CPU. That boundary defines what this release accelerates, and a published network model is what established where it lies (Section 1).

The correctness and construction fixes reach further than the accelerator does. The three Hines-solver defects silently suppressed `inject`-driven voltage transients for every neuron but the first in any multi-neuron `hsolve`, and the five linear-scan patterns behind $O(n^2)$ construction (Section 3) blocked population-scale models outright. Both predate this release and are fixed for every GENESIS/PGENESIS user, whether or not an accelerator is available.

5. Conclusions

   GENESIS 2.5 adds opt-in OpenCL and CUDA acceleration to GENESIS 2.4/PGENESIS 2.4 for Hodgkin–Huxley `tabchannel` models, without changes to model files, SLI scripts, or MPI workflows (Section 3 reports speedup and fidelity on cluster-class hardware). Beyond the single-compartment multiloop kernel, `hines_tree_eliminate` extends acceleration to axially-coupled multi-compartment trees at population scale; the single-detailed-cell regime is not helped by this design, which we measure and report.

   The release also carries seven correctness fixes to code already in users' hands. Three are Hines-solver defects that silently suppressed `inject`-driven voltage transients in any multi-neuron `hsolve`, and benefit any

GENESIS/PGENESIS user, accelerator or not; four are in the accelerator as first released (Section 1). Separately, five pre-existing linear-scan patterns compounding into $O(n^2)$ model-construction time are fixed, restoring linear scaling and making population-scale models buildable at all.

Measured against other simulators on a published network model, GENESIS 2.5 is 2.30× faster than CoreNEURON and 2.88× faster than NEURON 9.0.2 for single-threaded CPU execution of the Vogels–Abbott COBAHH benchmark, after verifying that the two implementations are equivalent (Section 3). That benchmark is a spiking network, which this release accelerates for the first time though only to parity with its own CPU arm, so CoreNEURON on an A100 still runs it 1.23× faster. On multi-compartment populations, where the accelerator does apply, GENESIS 2.5 overtakes the GPU-native Arbor beyond 64 ms of simulated time.

Future work. Four directions follow directly from the limits measured here, in what we judge to be descending order of value to users.

Scale. The kernel assigns one thread per tree, so it accelerates population-scale simulation and not the single detailed cell, where only one thread is active and the GPU is ∼20× slower than the CPU. Intra-tree parallelism, such as cyclic reduction, would extend acceleration to the detailed-morphology regime much of the literature works in. This is where the present design is furthest from complete.

Coverage. The accelerator now runs a synaptically coupled spiking network. GENESIS allowed one spike generator per `hsolve`, so such a model was built as one solver per cell; lifting that lets Vogels–Abbott be built as one solver per layer, 1.56× faster on CPU alone, the device reproducing it to 0.35% on spike count. A zero-delay network cannot batch, so every step pays a dispatch, which outweighs what the kernel saves: the GPU arm is 1.09× slower. Enlarging it does not help: at 12 500 cells, with the in-degree held so the 27 Hz regime survives, the arms are level (127.1 ± 1.7 s against 130.7 ± 2.5). The accelerable fraction stays near 59% at any size, so the ceiling stays 2.4× and the dispatch spends it. Reaching the profile's 2.4× ceiling means cutting the dispatch, not growing the model.

Comparison. CoreNEURON's multi-rank MPI configuration is still unmeasured and would place GENESIS 2.5 more completely.

Precision. The kernels are fp32 throughout, which lets them run on integrated GPUs lacking double precision at a cost of $\sim 10^{-7}$ V against the fp64 CPU. Whether that divergence stays bounded over the much longer runs used in plasticity studies is not established here.

Packaging into a container-based deployment pipeline [17] is in progress.

Acknowledgements

The authors thank the Maria Curie-Skłodowska University (UMCS) in Lublin and the LubMAN UMCS computing centre for access to the "Lunar" cluster (NVIDIA A100 and A40 GPU nodes) at the UMCS Institute of Mathematics, commissioned in 2015 [18] and since upgraded with the GPU nodes used in this work, and Karol S. Karpowicz and Prof. Przemysław Stpiczyński for provisioning and support of the compute environment on which the CUDA backend was built and validated. The authors also thank WarsawIQ for the AMD Radeon 890M integrated GPU on which the OpenCL backend was developed.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Anthropic's Claude to draft and edit the manuscript, to write the benchmarking and analysis scripts, and to assist with implementing and debugging the accelerator backends. All reported results were produced by running the software on the hardware named in the text and checked against the raw data in the repository. The authors reviewed and edited all output and take full responsibility for the content of the published article.

References

[1] Bower JM, Beeman D. *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural SImulation System.* Springer, 1998.

[2] GENESIS 3 Development Plans. `http://www.genesis-sim.org/GENESIS/G3/G3-dev-plans.html`

[3] Cornelis H, Coop AD, Rodriguez AL, Beeman D, Bower JM. GENESIS 3.0 functionality enhanced by interface to multiple types of database. *BMC Neuroscience* 2011, 12(Suppl 1):P15.

[4] Cornelis H, Edwards M, Coop AD, Bower JM. The CBI architecture for computational simulation of realistic neurons and circuits in the GENESIS 3 software federation. *BMC Neuroscience* 2008, 9(Suppl 1):P88.

[5] Cornelis H, Lu H, Esquivel A, Bower JM. Modeling a single dendritic compartment using Neurospaces and GENESIS-3. *BMC Neuroscience* 2007, 8(Suppl 2):P3.

[6] Cornelis H, Bampasakis D, Steuber V, Bower JM. Interoperability in the GENESIS 3.0 Software Federation: the NEURON Simulator as an Example. *BMC Neuroscience* 2013, 14(Suppl 1):P33.

[7] Akar NA, Cumming B, Karakasis V, Küsters A, Klijn W, Peyser A, Yates S. Arbor – A Morphologically-Detailed Neural Network Simulation Library for Contemporary High-Performance Computing Architectures. In: *27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*, 2019, pp. 274–282. doi:10.1109/EMPDP.2019.8671560.

[8] Kumbhar P, Hines M, Fouriaux J, Ovcharenko A, King J, Delalondre F, Schürmann F. CoreNEURON: An Optimized Compute Engine for the NEURON Simulator. *Frontiers in Neuroinformatics* 2019, 13:63. doi:10.3389/fninf.2019.00063.

[9] Markram H, Muller E, Ramaswamy S, et al. Reconstruction and Simulation of Neocortical Microcircuitry. *Cell* 2015, 163(2):456–492. doi:10.1016/j.cell.2015.09.029.

[10] Hines M. Efficient computation of branched nerve equations. *International Journal of Bio-Medical Computing* 1984, 15(1):69–76. doi:10.1016/0020-7101(84)90008-4.

[11] Hines ML, Carnevale NT. The NEURON Simulation Environment. *Neural Computation* 1997, 9(6):1179–1209. doi:10.1162/neco.1997.9.6.1179.

[12] Gewaltig M-O, Diesmann M. NEST (NEural Simulation Tool). *Scholarpedia* 2007, 2(4):1430. doi:10.4249/scholarpedia.1430.

[13] Stimberg M, Brette R, Goodman DFM. Brian 2, an intuitive and efficient neural simulator. *eLife* 2019, 8:e47314. doi:10.7554/eLife.47314.

[14] Yavuz E, Turner J, Nowotny T. GeNN: a code generation framework for accelerated brain simulations. *Scientific Reports* 2016, 6:18854. doi:10.1038/srep18854.

[15] Vogels TP, Abbott LF. Signal Propagation and Logic Gating in Networks of Integrate-and-Fire Neurons. *Journal of Neuroscience* 2005, 25(46):10786–10795. doi:10.1523/JNEUROSCI.3508-05.2005.

[16] Brette R, Rudolph M, Carnevale T, Hines M, Beeman D, Bower JM, et al. Simulation of networks of spiking neurons: A review of tools and strategies. *Journal of Computational Neuroscience* 2007, 23(3):349–398. doi:10.1007/s10827-007-0038-6.

[17] Chlasta K, Sochaczewski P, Wójcik GM, Krejtz I. Neural simulation pipeline: Enabling container-based simulations on-premise and in public clouds. *Frontiers in Neuroinformatics* 2023, 17:1122470. doi:10.3389/fninf.2023.1122470.

[18] Maria Curie-Skłodowska University (UMCS), Instytut Matematyki. Superkomputer LUNAR uruchomiony w Instytucie Matematyki UMCS. `https://www.umcs.pl/pl/aktualnosci,57,superkomputer-lunar-uruchomiony-w-instytucie-matematyki-umcs,27879.chtm`, 2015.

Current executable software version

Not applicable; this release is distributed as source code only (see Current code version above).